COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	September 5, 2026	**DATE SUBMITTED:**	September 8, 2026
NAME:	Silao, Johrel Louie T.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _7.1_ Sorting Algorithms Part 1

1. PROCEDURE OUTPUT

Code + Console Screenshot

```cpp
#include <iostream>
#include<cstdlib>
#include<ctime>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main() {

    // Create a array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];
    // Use time (0) as seed for random values
    srand(time(0));
    // Populate the array with random values
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand()%100;
    }
    // Display the randomly generated array
    std::cout << "Unsorted Array: \n";
    displayArr(unsortedArr);

    /*std::cout << "\nBubble Sort: \n";
    bubbleSort(unsortedArr, maxSize);
    displayArr(unsortedArr);*/
}

void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```
Unsorted Array:
81 13 22 13 29 26 66 73 16 22 18 81 58 69 78 32 90 35 0 94 63 20 32 73 92 92 64 21 32 25 9
5 70 75 54 49 57 91 85 22 16 0 15 1 60 38 47 31 76 94 68 2 46 72 75 31 65 1 85 44 48 31 3
96 73 90 87 14 59 22 99 66 86 59 52 86 35 95 28 64 11 5 62 7 90 8 29 72 41 74 95 86 10 89
77 90 94 76 10 99 91
```

Observations	This populates the 100 value integer array with random values using the rand()%100 and outputs the random generated integer.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```cpp
#include <iostream>
#include<cstdlib>
#include<ctime>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main() {

    // Create a array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];
    // Use time (0) as seed for random values
    srand(time(0));
    // Populate the array with random values
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand()%100;
    }
    // Display the randomly generated array
    std::cout << "Unsorted Array: \n";
    displayArr(unsortedArr);

    std::cout << "\nBubble Sort: \n";
    bubbleSort(unsortedArr, maxSize);
    displayArr(unsortedArr);
}

void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```cpp
#include <cstddef>
#include <iostream>
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

template<typename T>
void bubbleSort(T arr[], size_t arrSize) {
    // Step 1: for i = 0 to N - 1 repeat inner loop
    for (int i = 0; i < arrSize; i++) {
        // Step 2: Inner loop that forK = i + 1 to N - 1 repeat
        for (int j = i + 1; j < arrSize; j++) {
            // Step 3: Swap if the arr[j] > arr [i]
            if (arr[i] < arr[j]) {
                std::swap(arr[i], arr[j]);
            }
        }
    }
}

#endif
```

```
Unsorted Array:
87 50 51 95 43 82 28 89 42 44 0 2 92 86 69 86 13 52 82 11 33 2 20 38 93 92 79 85 9 5 69 61
 14 50 60 88 55 22 43 79 59 48 3 96 61 19 37 50 71 34 84 2 80 99 32 14 57 72 32 55 14 59 9
5 0 46 60 70 96 69 22 84 46 8 48 36 53 37 20 97 78 32 74 59 22 73 27 93 75 6 69 40 3 54 43
 19 43 59 36 34 92

Bubble Sort:
99 97 96 96 95 95 93 93 92 92 92 89 88 87 86 86 85 84 84 82 82 80 79 79 78 75 74 73 72 71
70 69 69 69 69 61 61 60 60 59 59 59 59 57 55 55 54 53 52 51 50 50 50 48 48 46 46 44 43 43
43 43 42 40 38 37 37 36 36 34 34 33 32 32 32 28 27 22 22 22 20 20 19 19 14 14 14 13 11 9 8
 6 5 3 3 2 2 2 0 0
```

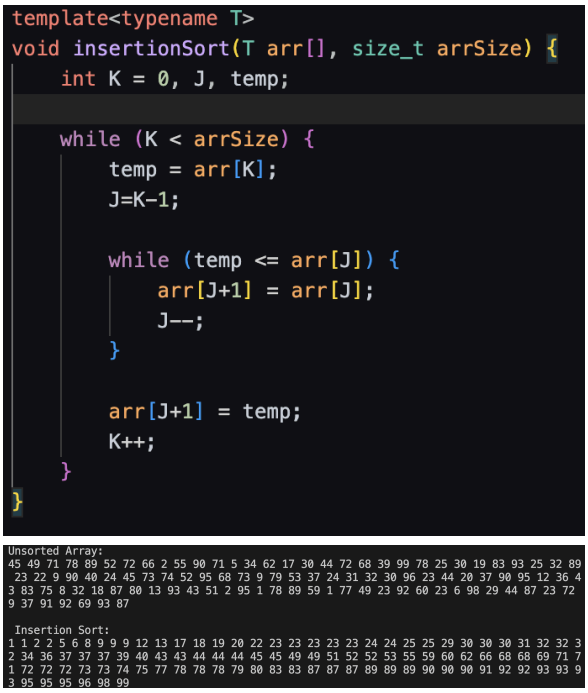
Observations	This implementation populates a 100 element integer array with random values and applies the template of bubblesort function from the header file. This sorts the array in a descending order.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observations	This program uses a helper function called routineSmallest() to continuously locate the index of the minimum element in the remaining unsorted portion of the array and swaps it to the current target which is the selectionSort(). This demonstrated the ascending order of the values.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	

	
Observations	This program implements the insertion sort using a while loop that sorts larger elements to the right before inserting the target element into the correct position.

```
template<typename T>
void insertionSort(T arr[], size_t arrSize) {
    int K = 0, J, temp;

    while (K < arrSize) {
        temp = arr[K];
        J=K-1;

        while (temp <= arr[J]) {
            arr[J+1] = arr[J];
            J--;
        }

        arr[J+1] = temp;
        K++;
    }
}
```

```
Unsorted Array:
45 49 71 78 89 52 72 66 2 55 90 71 5 34 62 17 30 44 72 68 39 99 78 25 30 19 83 93 25 32 89
 23 22 9 90 40 24 45 73 74 52 95 68 73 9 79 53 37 24 31 32 30 96 23 44 20 37 90 95 12 36 4
3 83 75 8 32 18 87 80 13 93 43 51 2 95 1 78 89 59 1 77 49 23 92 60 23 6 98 29 44 87 23 72
9 37 91 92 69 93 87

 Insertion Sort:
1 1 2 2 5 6 8 9 9 9 12 13 17 18 19 20 22 23 23 23 23 23 24 24 25 25 29 30 30 31 32 32 3
2 34 36 37 37 37 39 40 43 43 44 44 44 45 45 49 49 51 52 52 53 55 59 60 62 66 68 68 69 71 7
1 72 72 72 73 73 74 75 77 78 78 78 79 80 83 83 87 87 87 89 89 89 90 90 90 91 92 92 93 93 9
3 95 95 95 96 98 99
```

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Pseudocode of Algorithm

```
START
  1. Define array votes[100]
  2. Define candidateNames = [
       "Bo Dalton Capistrano",
       "Cornelius Raymon Agustin",
       "Deja Jayla Banaga",
       "Lalla Brielle Yabut",
       "Franklin Relano Castro"
     ]
  3. Define counts[5] initialized to 0

  4. Populate votes[0...99] with random integers from 1 to 5:
     FOR i = 0 TO 99 DO:
        votes[i] = (rand() % 5) + 1
     END FOR

  5. Sort the votes array using Insertion Sort:
     FOR k = 1 TO 99 DO:
        temp = votes[k]
        j = k - 1
        WHILE j >= 0 AND votes[j] > temp DO:
           votes[j + 1] = votes[j]
           j = j - 1
        END WHILE
        votes[j + 1] = temp
     END FOR

  6. Count frequencies of each candidate ID (1 to 5):
     FOR i = 0 TO 99 DO:
        candidateID = votes[i]
        counts[candidateID - 1] = counts[candidateID - 1] + 1
     END FOR

  7. Determine Winner:
     maxVotes = 0
     winningIndex = 0
     FOR c = 0 TO 4 DO:
        IF counts[c] > maxVotes THEN:
           maxVotes = counts[c]
```

```
            winningIndex = c
        END IF
    END FOR

    8. Display sorted array, vote tallies per candidate, and the winner.
END
```

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "ilob_Supplementary_Header.h"

const int total_votes = 100;

int main() {
    int votes[total_votes];
    int counts[6] = {0};

    std::string candidates[] = {
        "",
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Bañaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    srand(time(0));

    for (int i = 0; i < total_votes; i++) {
        votes[i] = (rand() % 5) + 1;
    }

    // Sorting of votes array
    insertionSort(votes, total_votes);

    // Counting of votes sequentially
    for (int i = 0; i < total_votes; i++) {
        counts[votes[i]]++;
    }

    // Display the Sorted Array
    std::cout << "=======================================" << std::endl;
    std::cout << "Sorted Array of Votes:\n";
    for (int i = 0; i < total_votes; i++) {
        std::cout << votes[i] << " ";
        if ((i + 1) % 20 == 0) std::cout << "\n";
    }
    std::cout << "=======================================" << std::endl;

    // Display Vote Counts and check who is the  Winner
    std::cout << "\n ============== ELECTION RESULTS ============== \n";
    int maxVotes = -1;
    int winnerID = 1;

    for (int i = 1; i <= 5; i++) {
        std::cout << "Candidate " << i << " (" << candidates[i] << "): " << counts[i] << " votes\n";
        if (counts[i] > maxVotes) {
            maxVotes = counts[i];
            winnerID = i;
        }
    }

    std::cout << "\nWINNING CANDIDATE: " << candidates[winnerID] << " with " << maxVotes << " votes!\n";
    std::cout << "=======================================";

    return 0;
}
```

Figure 1: Code Snippet of main.cpp

Observations of Figure 1:

- This main program initializes an array of 100 random generated integers that ranges from 1 - 5 and calls the insertionSort() to organize the votes in order. Counts and evaluates the total votes for each candidate using an array index accumulator, and eventually displays the winner.

```
#include <cstddef>
#include <iostream>
#ifndef ILOB_SUPPLEMENTARY_HEADER_H
#define ILOB_SUPPLEMENTARY_HEADER_H

template<typename T>
void insertionSort(T arr[], int size) {
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

#endif
```

Figure 2: Code Snippet of ilob_supplementary_header.h

Observation of Figure 2:

- This header file contains the function that places each vote key into its correct position.

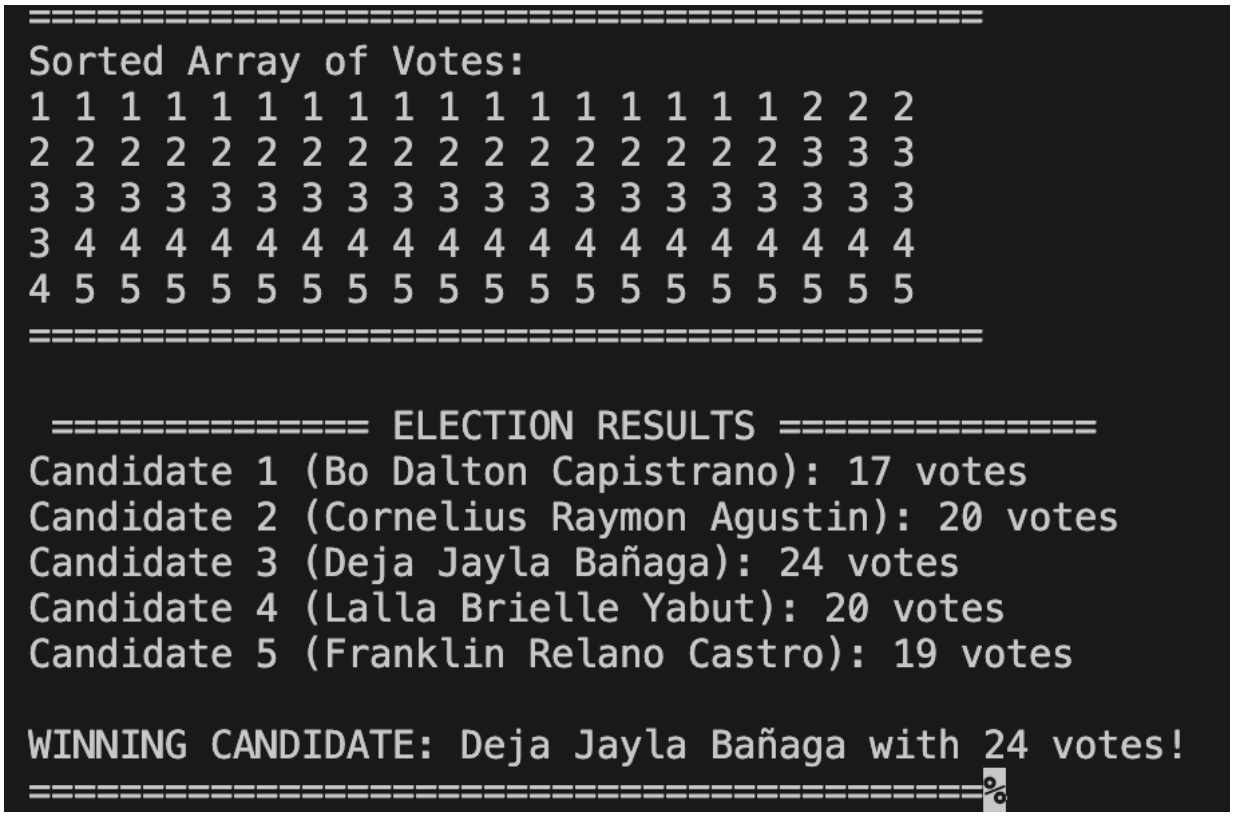
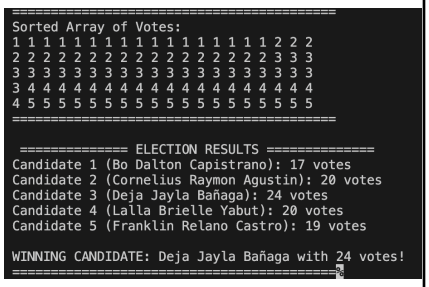


Figure 3: Output Snippet of main.cpp

Observation of Figure 3:

- This terminal displays the expected output of all the 100 sorted values followed by the detailed candidate votes and its names, and displays the winning candidate of the program.

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	17 counts of **Candidate 1**	17
	20 counts of **Candidate 2**	20
	24 counts of **Candidate 3**	24
	20 counts of **Candidate 4**	20
	19 counts of **Candidate 5**	19

Question: Was your developed vote counting algorithm effective? Why or why not?

- Yes, the implementation of the algorithm was effective because it sorts the array on counting the random generated data. However, sorting the data isn't strictly mandatory for evaluating the small values, adding Insertion Sort handles the given problem perfectly, verifying data accuracy easily through visual inspection of continuous values and producing precise election results.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.